

Aula 2 - App JokenPo em Flutter

Parte 1

Objetivos da aula:

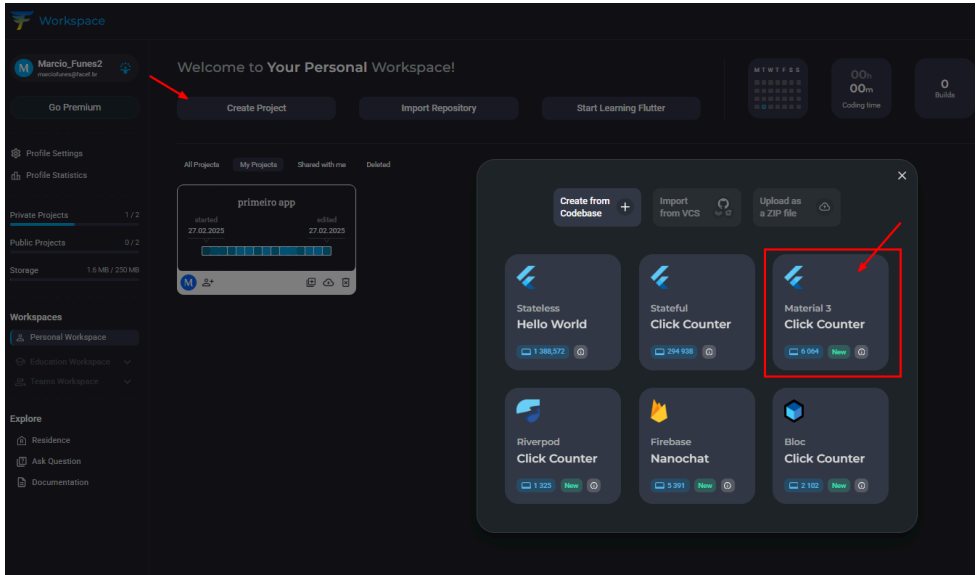
- Criando a estrutura do App JokenPo..... 2
- Importando o Material Design..... 5
- Criando uma nova classe..... 10
- Definindo a rota com a propriedade Home..... 13
- Utilizando o Scaffold na classe Jogo..... 14
- Utilizando o Child e Children..... 19
- Adicionando imagens ao body..... 23
- Editando o pubspec.yaml..... 27
- Código-fonte completo..... 32

Criando a estrutura do App JokenPo

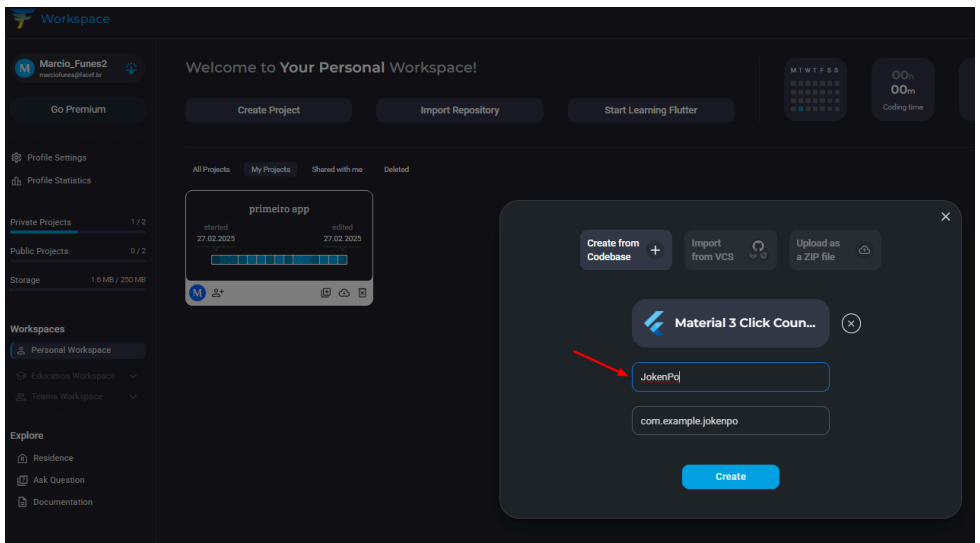
1 - Visando melhor compreensão sobre os componentes necessários para o desenvolvimento de uma aplicação mobile, vamos criar um App que permite ao usuário jogar JokenPo contra o aplicativo:



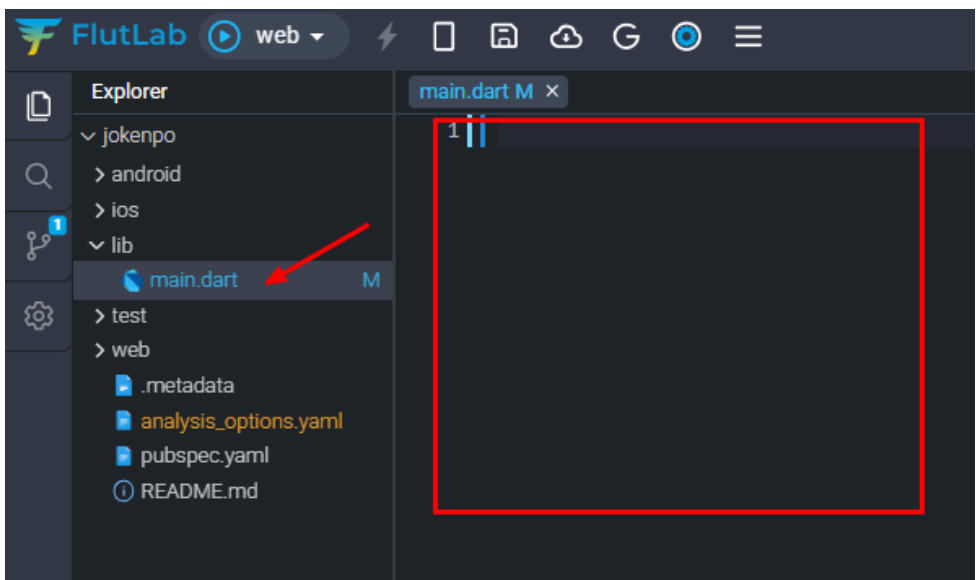
2 - Vamos iniciar um novo projeto, acesse o <https://flutlab.io/workspace> clique em **Create Project** e vamos escolher um template contendo o Material 3 e assim utilizar em nosso projeto.



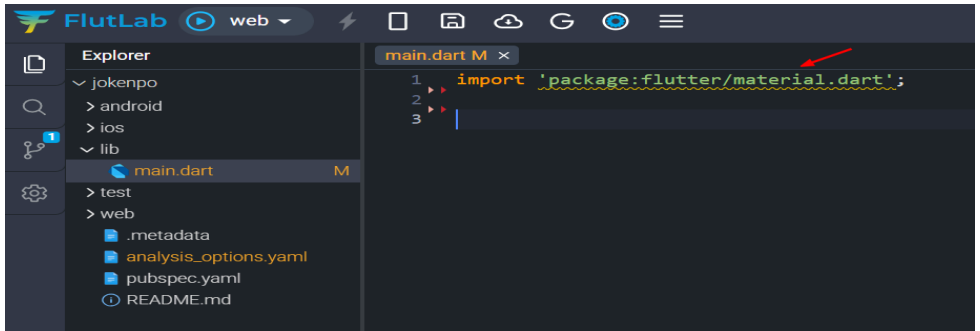
3 - Coloque o nome do projeto como JokenPo e clique em **Create**.



4 - Como desejamos criar a aplicação do zero, apague todo o conteúdo do arquivo main.dart.



5 - Para iniciar um novo app precisamos importar no Dart algum pacote de funcionalidades que irão nos ajudar no desenvolvimento. Nesse app iremos usar o Material Design, faça a importação como na linha 1:



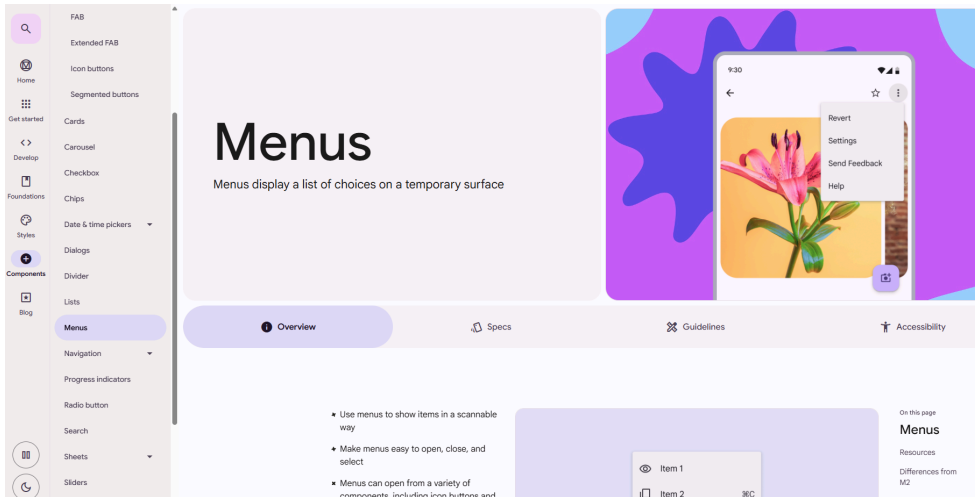
Importando o Material Design

6 - O **Material Design** é um sistema adaptável de diretrizes, componentes e ferramentas que dão suporte às melhores práticas de design de interface de usuário definidos pelo Google. Em outras palavras, é um Guideline que o Google recomenda para desenvolvimento Web e Mobile.

Veja a documentação: <https://m3.material.io/components/buttons/overview>

É muito comum as empresas adotarem alguma padronização de mercado para que o usuário possa reconhecer os elementos de UI e assim

melhorar sua experiência. No exemplo abaixo temos as definições de Menus para aplicações mobile.



7 - O primeiro passo é criarmos uma função main para começar a instanciar funções e recursos da aplicação:

```
main.dart x
1  import 'package:flutter/material.dart';
2
3  void main() {
4
5
6  }
7
```

8 - Usar um framework ou biblioteca significa aumentar a velocidade de desenvolvimento ao utilizarmos códigos já existentes nestes frameworks ou bibliotecas. Em nosso caso vamos usar um método já existente no Flutter chamado `runApp()` que é responsável por iniciar o aplicativo Flutter e anexar o widget raiz à tela. Por isso a importância de estudar a documentação oficial e melhor entender os conceitos da tecnologia:

<https://api.flutter.dev/flutter/widgets/runApp.html>

The screenshot shows a code editor with a file named `main.dart`. The code contains an `import` statement for `package:flutter/material.dart` and a `void main()` function. Inside the function, the `runApp` method is being typed, and a dropdown menu is visible. A red arrow points to the `runApp(...)` option in the menu. The menu also lists other Flutter-related classes and methods like `ResourceHandle.fromReadPipe(...)` and `RouteInformationReportingType`.

```
1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp
5   runApp(...) (Widget app) → void
6   ResourceHandle.fromReadPipe(...)
7   RouteInformationReportingType
8   RouteInformationReportingType.navigate
9   RouteInformationReportingType.neglect
10  RouteInformationReportingType.none
```

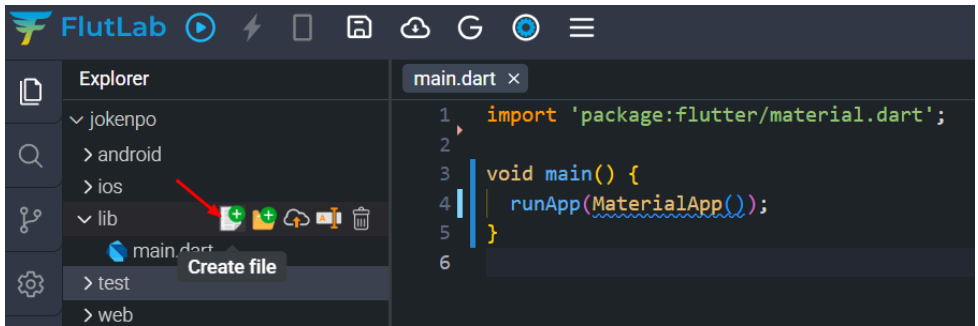
9 - Dentro do `runApp` vamos dizer ao Flutter que queremos usar as funcionalidades já existentes no Material Design chamando a função `MaterialApp()`.

Veja na documentação quais elementos essa função possibilita:

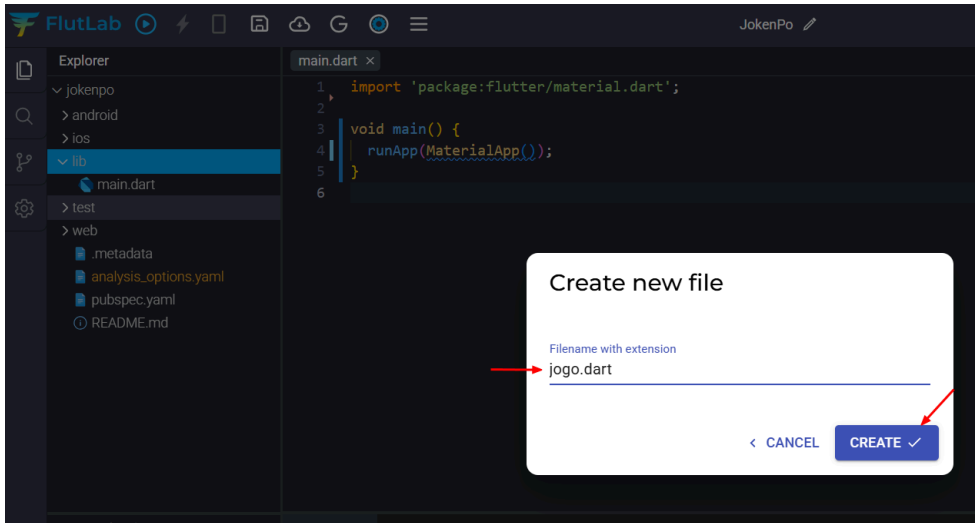
<https://api.flutter.dev/flutter/material/MaterialApp-class.html>

```
main.dart x
1  import 'package:flutter/material.dart';
2
3  void main() {
4    runApp(MaterialApp());
5  }
6
7
```

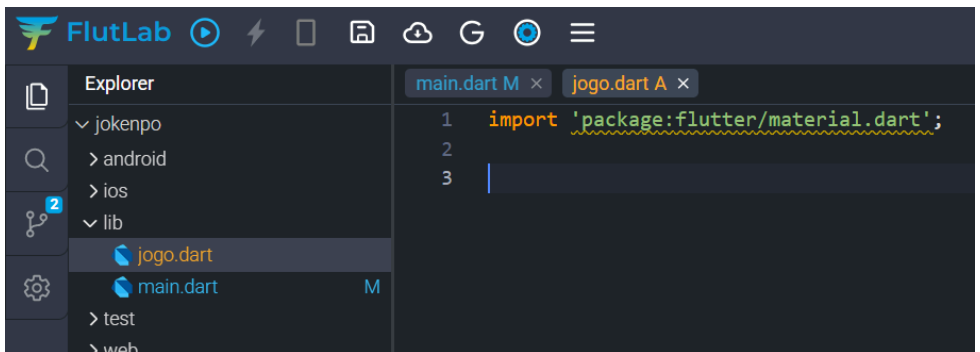
10 - Nem só de um arquivo Dart se faz uma aplicação, portanto, vamos começar a instanciar novas classes visando boas práticas e organização de código. Na pasta lib onde já temos o main.dart, clique em Create file:



11 - Dê o nome desse arquivo de “jogo.dart” e clique em Create:



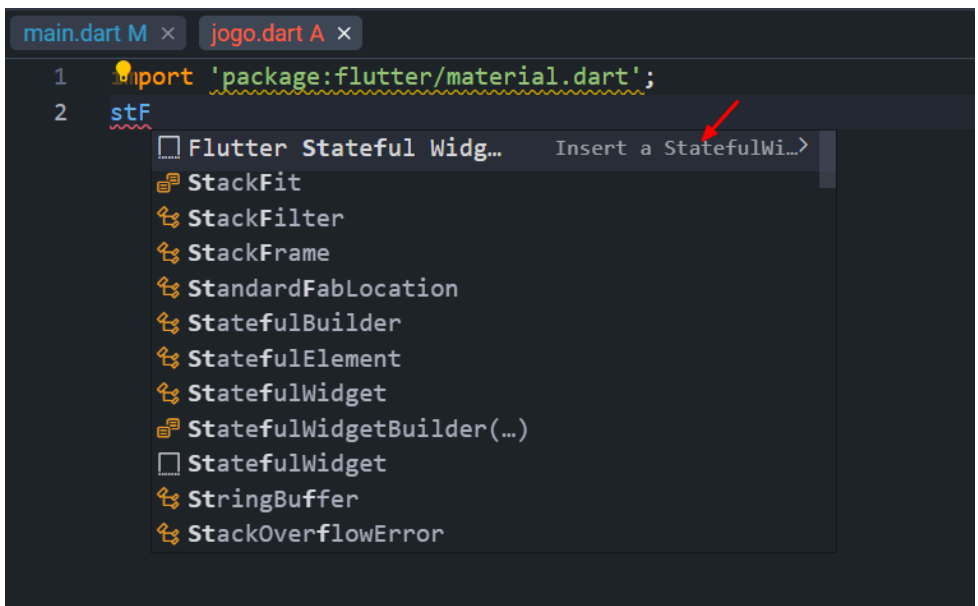
12 - Dentro de nosso novo arquivo Dart, vamos começar importando o Material Design:



Criando uma nova classe

13 - Para de fato termos uma nova classe que poderá ser instanciada dentro do main.dart vamos usar o StatefulWidget define Widgets que mudam suas características com a mudança de estado, ou seja, tem um estado mutável. Durante a execução do app, se a informação do estado mudar, isso será refletido no Widget.

Por exemplo, em um aplicativo ao clicar em um botão que é um StatefulWidget, um número é incrementado. O valor do número é uma informação ou estado que mudou e isso se refletiu no Widget Text que exibe o valor do número no aplicativo.



```
main.dart M x jogo.dart A x
1 import 'package:flutter/material.dart';
2 stF
```

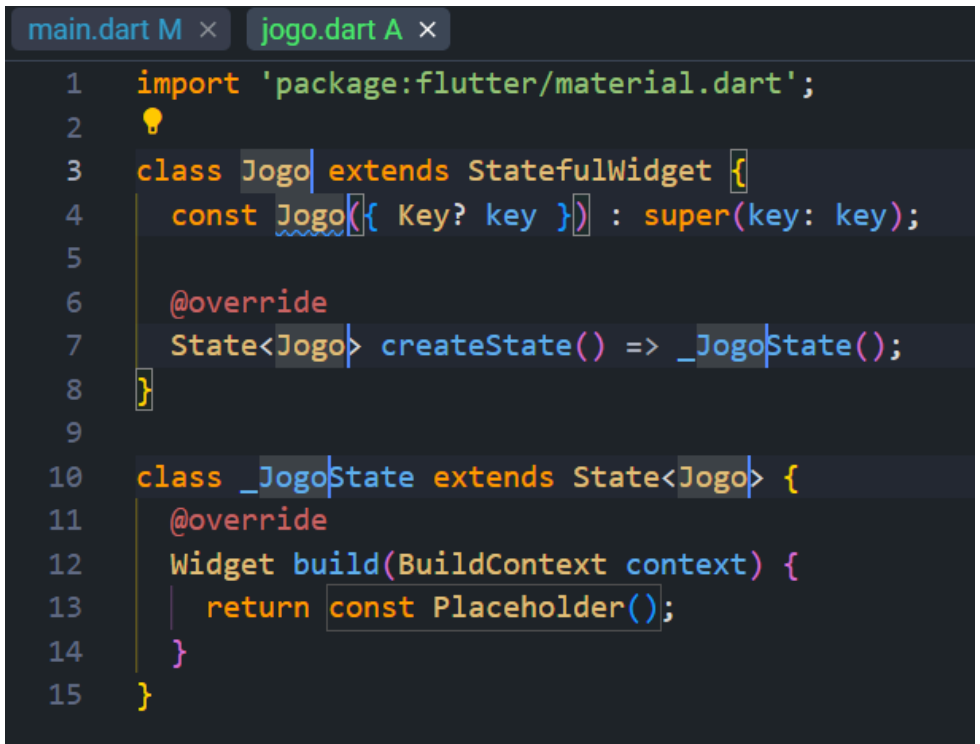
- Flutter StatefulWidget...
- StackFit
- StackFilter
- StackFrame
- StandardFabLocation
- StatefulBuilder
- StatefulElement
- StatefulWidget
- StatefulWidgetBuilder(...)
- StatefulWidget
- StringBuffer
- StackOverflowError

14 - Ao clicar no Insert a StatefulWidget o Flutter irá adicionar automaticamente a estrutura de classe do Dart:

```
main.dart M x  jogo.dart A x
1  import 'package:flutter/material.dart';
2
3  class extends StatefulWidget {
4      const ({ Key? key }) : super(key: key);
5
6      @override
7      State<> createState() => _State();
8  }
9
10 class _State extends State<> {
11     @override
12     Widget build(BuildContext context) {
13         return const Placeholder();
14     }
15 }
```

15 - Na linha 3 depois de class vamos dizer qual o nome de nossa nova classe, coloque Jogo, o editor irá automaticamente editar as linhas seguintes usando o nome Jogo, desse modo:

Dica: use o nome da classe com o mesmo nome do arquivo .dart. Nome da classe letra maiúscula e nome de arquivo letra minúscula:



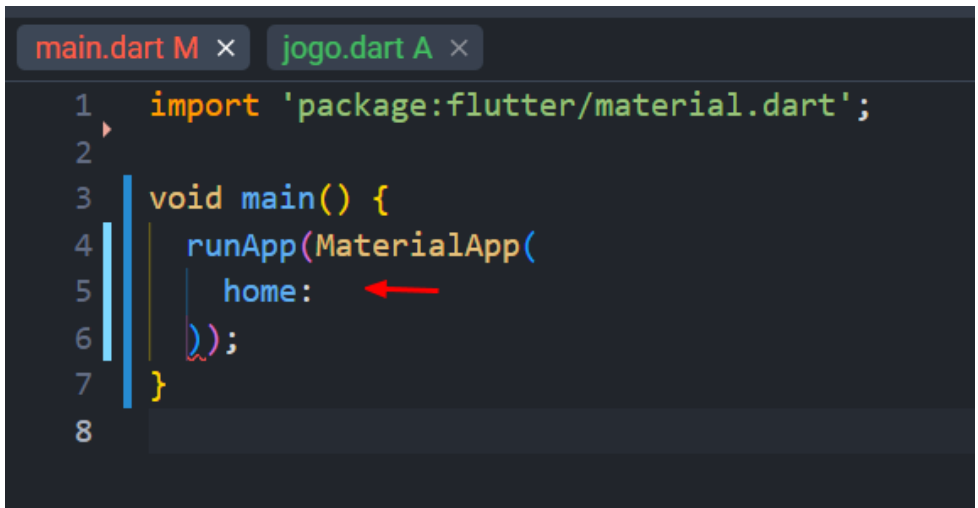
The screenshot shows an IDE with two tabs: 'main.dart M' and 'jogo.dart A'. The 'jogo.dart A' tab is active, displaying the following Dart code:

```
1  import 'package:flutter/material.dart';
2  ⚡
3  class Jogo extends StatefulWidget {
4      const Jogo({ Key? key }) : super(key: key);
5
6      @override
7      State<Jogo> createState() => _JogoState();
8  }
9
10 class _JogoState extends State<Jogo> {
11     @override
12     Widget build(BuildContext context) {
13         return const Placeholder();
14     }
15 }
```

Definindo a rota com a propriedade Home

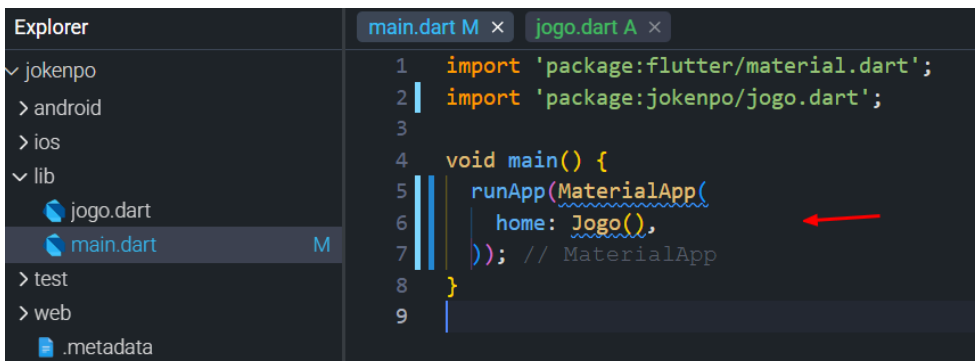
16 - Voltando para o main.dart, vamos adicionar dentro do MaterialApp() a propriedade home que define a rota que é exibida primeiro quando o aplicativo é iniciado normalmente. Ou seja, o que definirmos em home será aquilo que o usuário visualiza ao abrir a aplicação. Documentação:

<https://api.flutter.dev/flutter/material/MaterialApp/home.html>



```
main.dart M x  jogo.dart A x
1  import 'package:flutter/material.dart';
2
3  void main() {
4    runApp(MaterialApp(
5      home:  ←
6    ));
7  }
8
```

17 - Chame a classe Jogo() para a propriedade home:



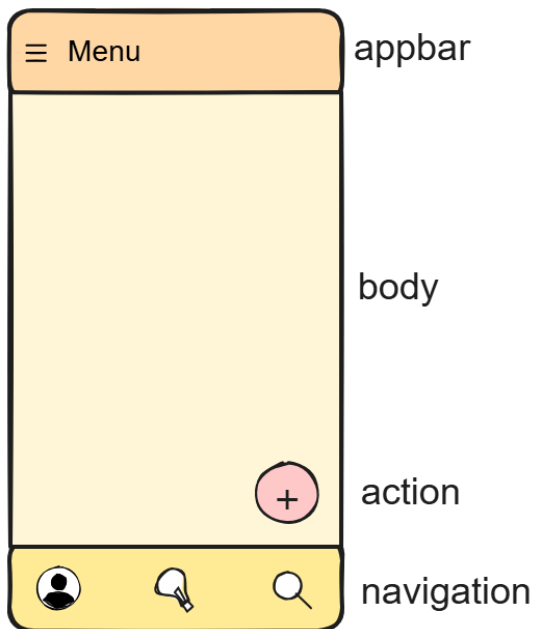
```
Explorer
└─ jokenpo
   └─ > android
   └─ > ios
   └─ lib
      └─ jogo.dart
      └─ main.dart M
      └─ > test
      └─ > web
      └─ .metadata

main.dart M x  jogo.dart A x
1  import 'package:flutter/material.dart';
2  import 'package:jokenpo/jogo.dart';
3
4  void main() {
5    runApp(MaterialApp(
6      home: Jogo(),  ←
7    )); // MaterialApp
8  }
9
```

Utilizando o Scaffold na classe Jogo

18 - Se você emulasse agora sua aplicação nada iria aparecer pois nossa classe `Jogo()` está vazia. Para mudar isso dentro do `Widget build` vamos retornar alguns elementos que deverão ser construídos na interface do usuário pela nossa classe.

A organização visual e a estruturação das telas são aspectos fundamentais de qualquer aplicativo Flutter. O widget `Scaffold` cumpre esta função. Ele fornece uma estrutura básica e que abrange toda a tela:



Exemplos: <https://flutterparainiciantes.com.br/scaffold/>

Vamos então utilizar o Scaffold (linha 13) e depois vamos criar as estruturas base da nossa aplicação, uma appBar e o body:

```
main.dart M x  jogo.dart A x
1  import 'package:flutter/material.dart';
2
3  class Jogo extends StatefulWidget {
4      const Jogo({Key? key}) : super(key: key);
5
6      @override
7      State<Jogo> createState() => _JogoState();
8  }
9
10 class _JogoState extends State<Jogo> {
11     @override
12     Widget build(BuildContext context) {
13         return Scaffold(
14             appBar: ,
15             body: ,
16         );
17     }
18 }
19
```

19 - Dentro de appBar (ou seja, a região do topo da aplicação) vamos colocar de fato um Widget AppBar (apenas uma barra que pode receber outros elementos). Dentro do Widget AppBar vamos definir uma cor para o fundo, uma para o texto e o texto em si:

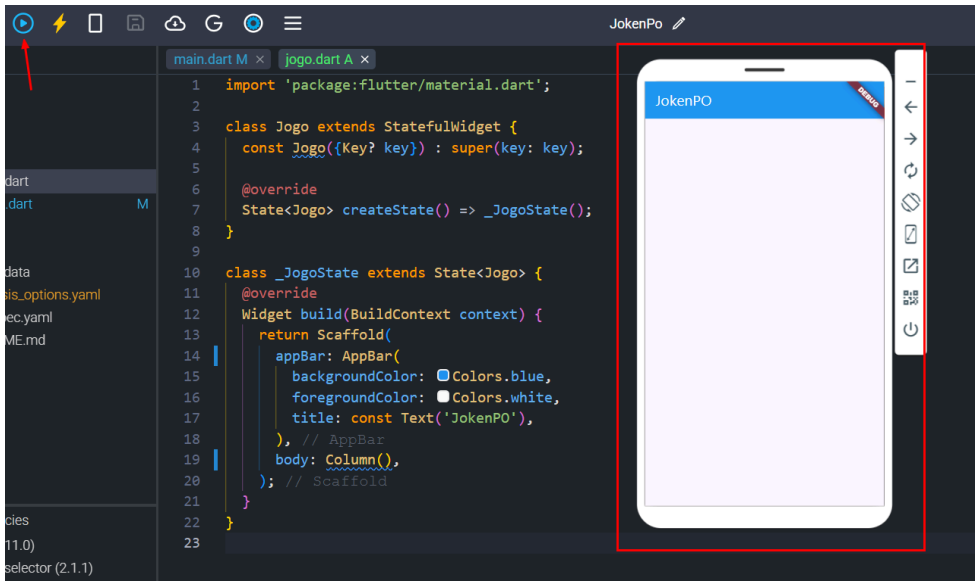
```
10  class _JogoState extends State<Jogo> {
11    @override
12    Widget build(BuildContext context) {
13      return Scaffold(
14        appBar: AppBar(
15          backgroundColor: Colors.blue,
16          foregroundColor: Colors.white,
17          title: const Text('JokenPO'),
18        ), // AppBar
19        body: ,
20      ); // Scaffold
21    }
22  }
23
```



20 - Para o body da aplicação vamos organizar os elementos em colunas, portanto, chame a função `Column()` como na linha 19:

```
main.dart M x  jogo.dart A x
1  import 'package:flutter/material.dart';
2
3  class Jogo extends StatefulWidget {
4      const Jogo({Key? key}) : super(key: key);
5
6      @override
7      State<Jogo> createState() => _JogoState();
8  }
9
10 class _JogoState extends State<Jogo> {
11     @override
12     Widget build(BuildContext context) {
13         return Scaffold(
14             appBar: AppBar(
15                 backgroundColor: Colors.blue,
16                 foregroundColor: Colors.white,
17                 title: const Text('JokenPO'),
18             ), // AppBar
19             body: Column(),
20         ); // Scaffold
21     }
22 }
23
```

21 - Vamos testar o uso do Scaffold, clique em Build Project (CTRL+B) e aguarde a emulação. Esse deverá ser o resultado atual:



Utilizando o Child e Children

22 - Para organizar os elementos presentes em nossa coluna, vamos entender as propriedades child (criança) e children (crianças) onde:

child pega um único widget

Exemplo: child: Text('foo')

children pegam uma lista de widgets

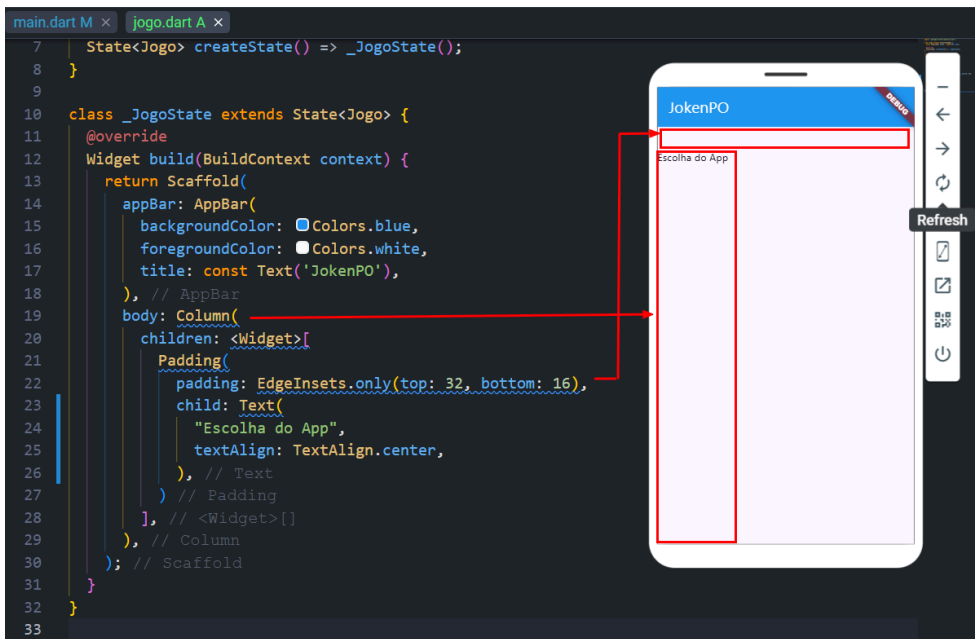
Exemplo: children: <Widget>[Text('foo'), Text('bar')]

Como vamos colocar diversas imagens e texto utilize o children para posicionar uma lista de elementos:

```
10 class _JogoState extends State<Jogo> {
11   @override
12   Widget build(BuildContext context) {
13     return Scaffold(
14       appBar: AppBar(
15         backgroundColor: Colors.blue,
16         foregroundColor: Colors.white,
17         title: const Text('JokenPO'),
18       ), // AppBar
19       body: Column(
20         children: <Widget>[
21           |
22         ], // <Widget>[]
23       ), // Column
24     ); // Scaffold
```



23 - Vamos adicionar um Padding dentro da nossa coluna, definir um espaçamento e adicionar um texto. Mesmo pedindo para centralizar na linha 25, perceba que como nossa coluna ainda não está centralizada o texto apenas centraliza dentro da coluna que está invisível alinhada a esquerda.



Veja mais exemplos na documentação:

<https://docs.flutter.dev/ui/widgets/layout>

24 - Dentro da coluna na linha 20 adicione um `crossAxisAlignment.stretch` que faz o que o elemento ocupe todo o espaço disponível no body:

```
10 class _JogoState extends State<Jogo> {
11   @override
12   Widget build(BuildContext context) {
13     return Scaffold(
14       appBar: AppBar(
15         backgroundColor: Colors.blue,
16         foregroundColor: Colors.white,
17         title: const Text('JokenPO'),
18       ), // AppBar
19       body: Column(
20         crossAxisAlignment: CrossAxisAlignment.stretch,
21         children: <Widget>[
22           Padding(
23             padding: EdgeInsets.only(top: 32, bottom: 16),
24             child: Text(
25               "Escolha do App",
26               textAlign: TextAlign.center,
27             ), // Text
28           ) // Padding
29         ], // <Widget>[]
30       ), // Column
31     ); // Scaffold
32   }
33 }
34
```



The screenshot shows a mobile application interface. At the top is a blue AppBar with the title 'JokenPO' in white. Below the AppBar is a light purple background with the text 'Escolha do App' centered. The code on the left shows the implementation of this interface using Flutter's Scaffold and Column widgets.

Column

CrossAxis Alignment

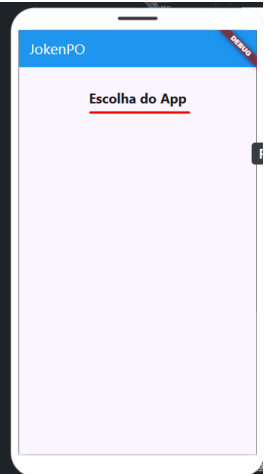
.center	.start	.end	.stretch
			

MainAxis Alignment

.center	.start	.end	.spaceEvenly	.spaceAround	.spaceBetween
					

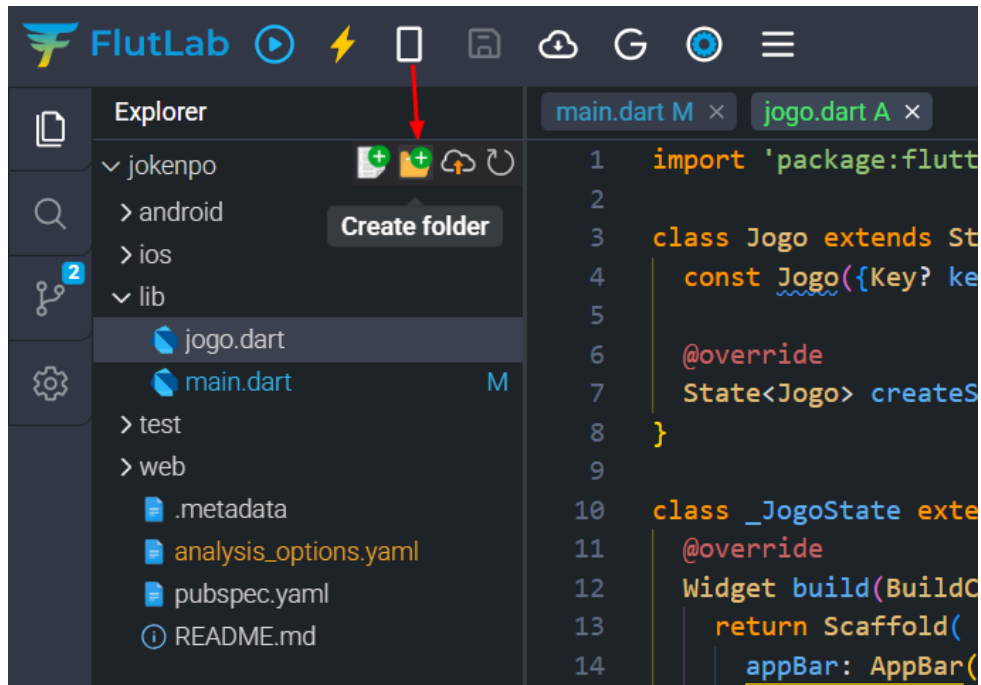
25 - Agora que estamos centralizados, vamos mudar o tamanho e o colocar em negrito. Deixei o código como abaixo:

```
10 class _JogoState extends State<Jogo> {
11   @override
12   Widget build(BuildContext context) {
13     return Scaffold(
14       appBar: AppBar(
15         backgroundColor: Colors.blue,
16         foregroundColor: Colors.white,
17         title: const Text('JokenPO'),
18       ), // AppBar
19       body: const Column(
20         crossAxisAlignment: CrossAxisAlignment.stretch,
21         children: <Widget>[
22           Padding(
23             padding: EdgeInsets.only(top: 32, bottom: 16),
24             child: Text(
25               "Escolha do App",
26               textAlign: TextAlign.center,
27               style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
28             ), // Text
29           ], // Padding
30         ], // <Widget>[]
31       ), // Column
32     ); // Scaffold
33   }
34 }
35
```

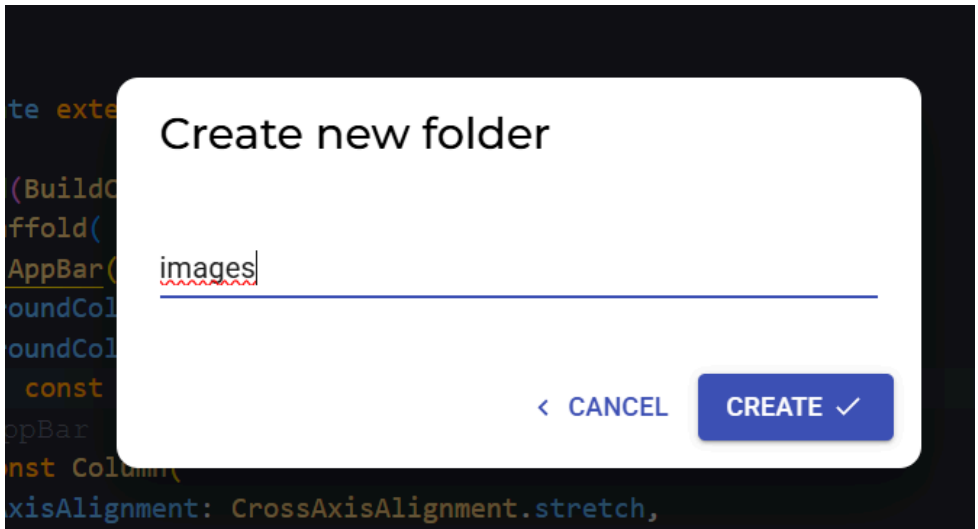


Adicionando imagens ao body

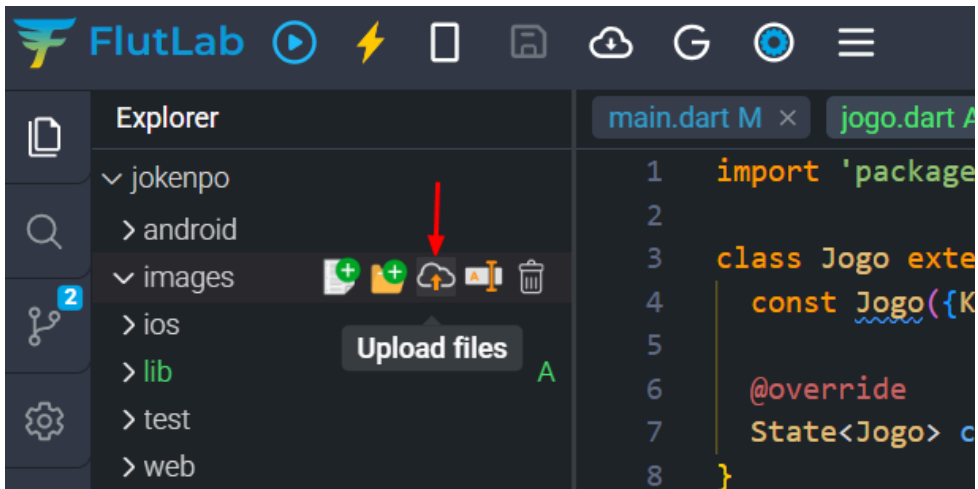
26 - Vamos criar uma pasta para importar algumas imagens. No projeto vá até a pasta jokenpo e clique em Create folder:



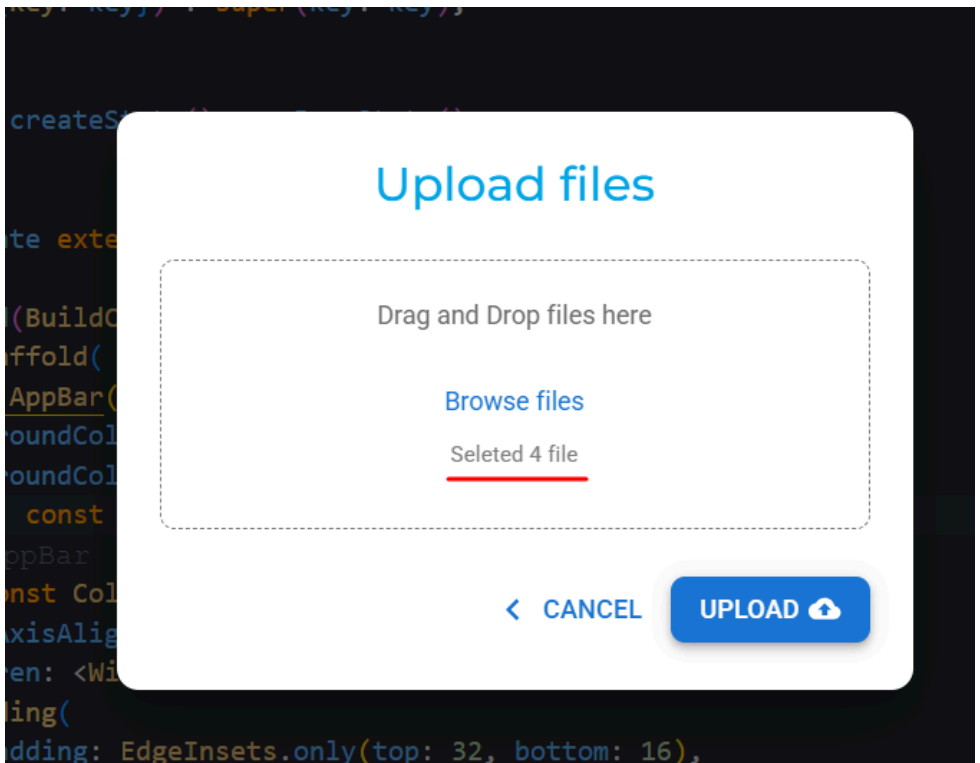
27 - Coloque o nome de “images” e clique em CREATE



28 - Clique em Upload files:

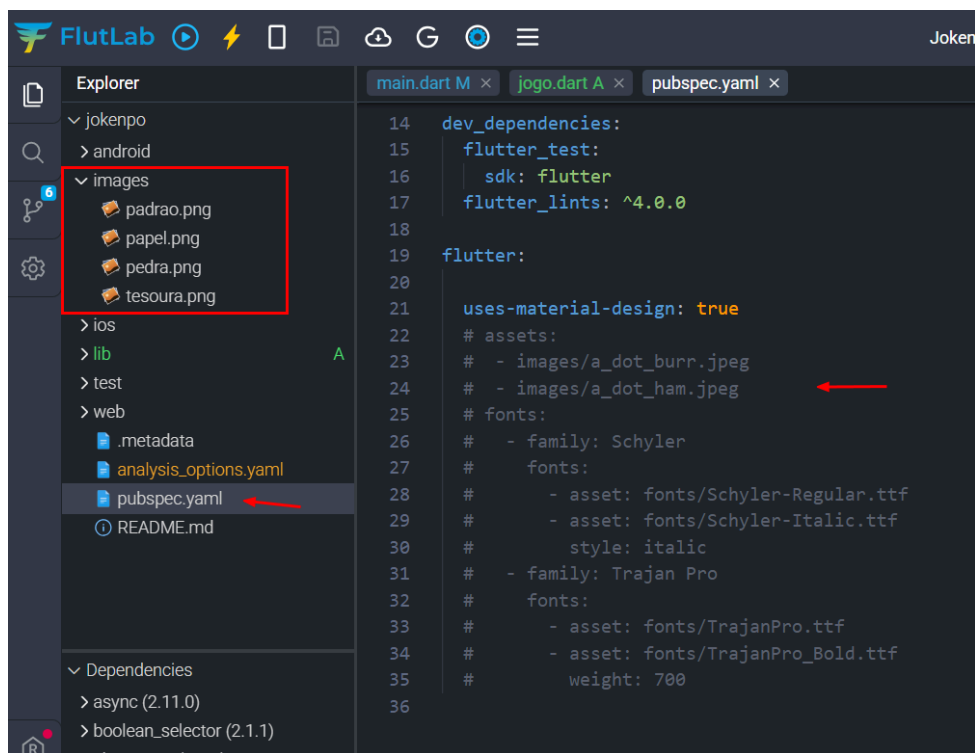


29 - Arraste as imagens para o Upload. As imagens estão no AVA:



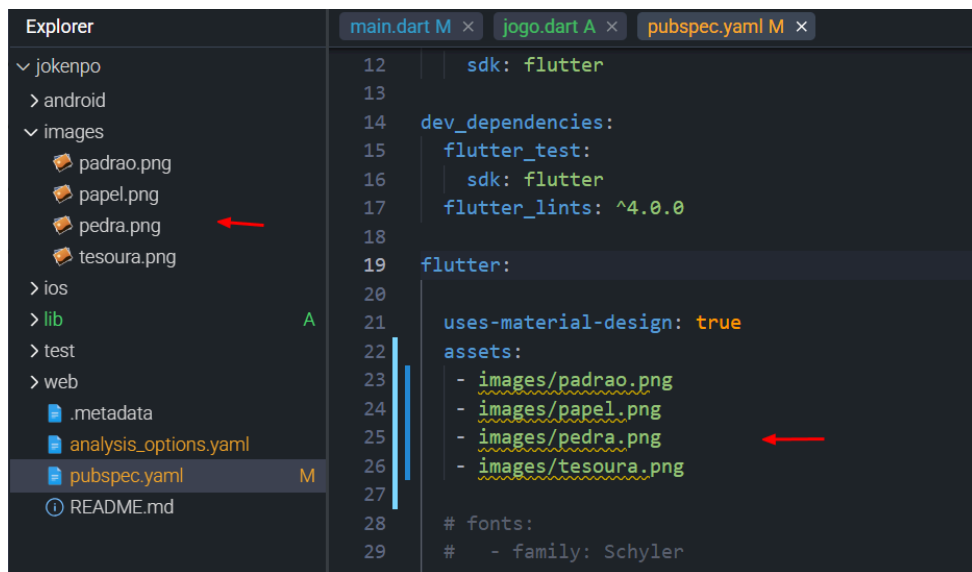
30 - Agora que importamos nossas 4 imagens para a pasta images. Localize o arquivo pubspec.yaml e abra este arquivo.

O arquivo pubspec especifica dependências que o projeto requer, como pacotes específicos (e suas versões), fontes ou arquivos de imagem. Ele também especifica outros requisitos, como dependências em pacotes de desenvolvedor (como pacotes de teste ou mocking) ou restrições específicas na versão do Flutter SDK.

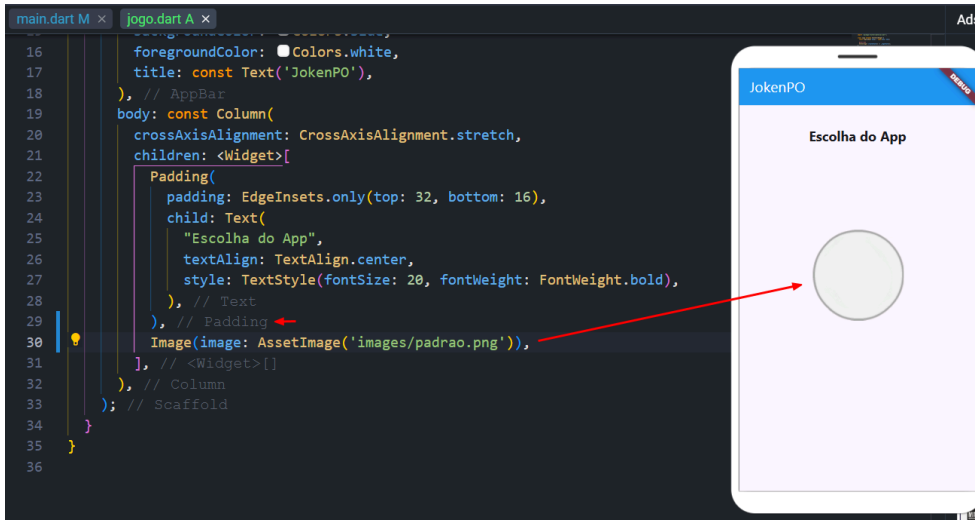


Editando o pubspec.yaml

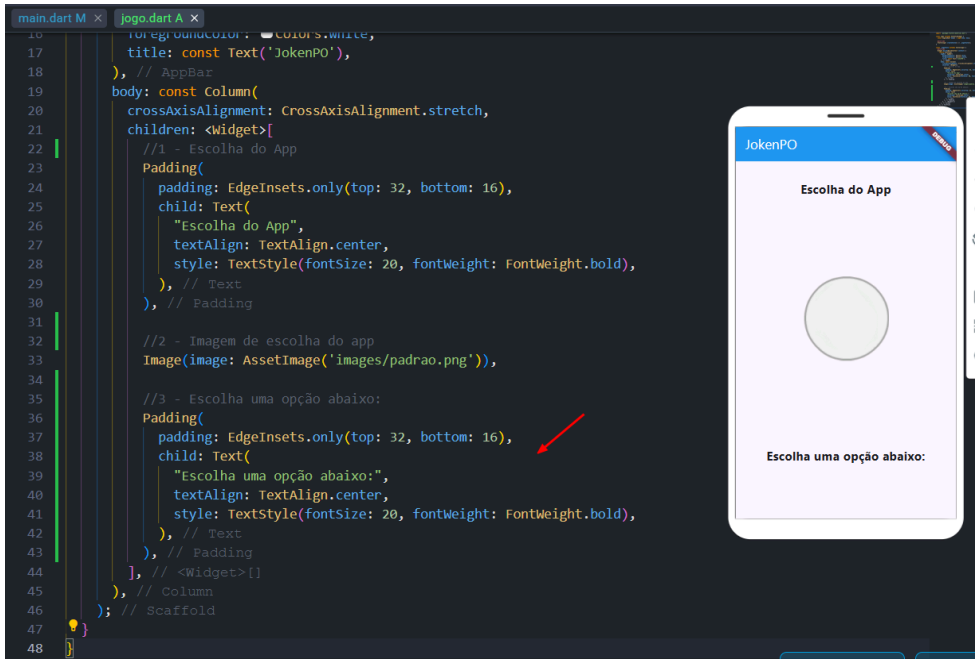
31 - Descomente as linhas referentes aos assets. Veja que é necessário passar o nome correto da pasta e o nome dos arquivos para que o Flutter possa permitir o correto referenciamento destes arquivos nas classes. Após editar os nomes corretamente, faça o build para o Web novamente (CTRL+B).



32 - Agora podemos utilizar sem problemas as imagens em nosso código. Coloque uma vírgula na linha 29 logo após o parenteses do Padding e chame a imagem.

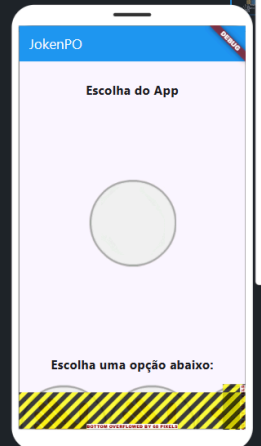


33 - Vamos criar mais um texto abaixo da imagem. Utilize os comentários para melhor organizar seu código:

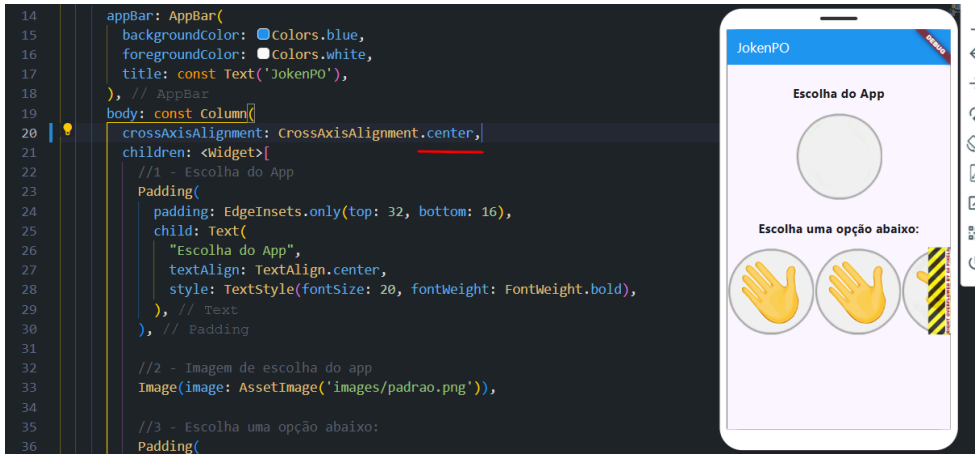


34 - Após a vírgula do Padding na linha 43, vamos criar uma linha que irá abrigar as outras 3 imagens. Veja que após realizar o build temos as linhas saindo para fora do body.

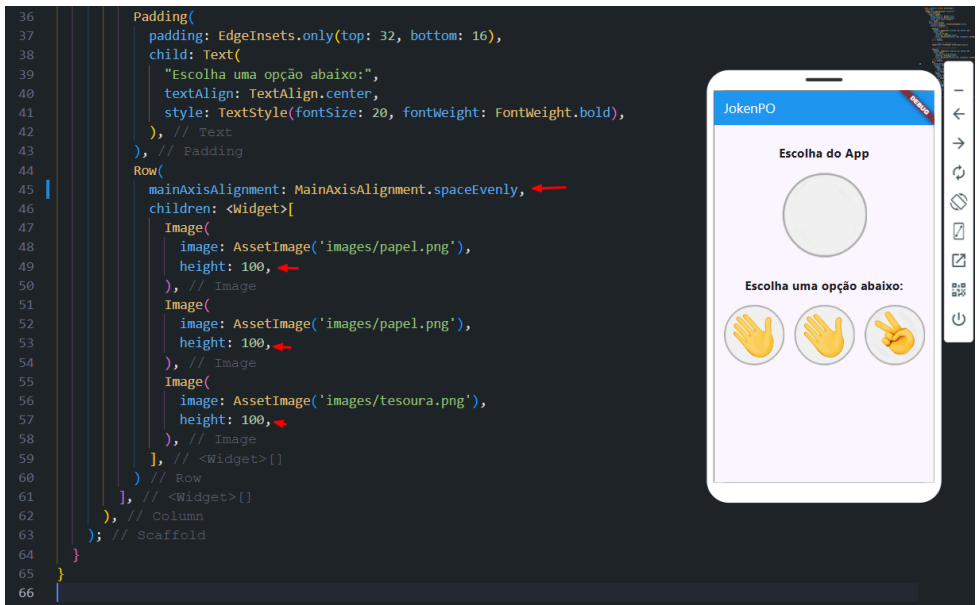
```
25     child: Text(  
26       "Escolha do App",  
27       textAlign: TextAlign.center,  
28       style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
29     ), // Text  
30   ), // Padding  
31  
32   //2 - Imagem de escolha do app  
33   Image(image: AssetImage('images/padrao.png')),  
34  
35   //3 - Escolha uma opção abaixo:  
36   Padding(  
37     padding: EdgeInsets.only(top: 32, bottom: 16),  
38     child: Text(  
39       "Escolha uma opção abaixo:",  
40       textAlign: TextAlign.center,  
41       style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
42     ), // Text  
43   ), // Padding  
44   Row(  
45     children: <Widget>[  
46       Image(image: AssetImage('images/papel.png')),  
47       Image(image: AssetImage('images/papel.png')),  
48       Image(image: AssetImage('images/tesoura.png')),  
49     ], // <Widget> []  
50   ) // Row  
51 ], // <Widget> []  
52 ), // Column  
53 ); // Scaffold  
54 }  
55 ]  
56
```



35 - Para corrigir isso mude o `CrossAxisAlignment` para `center` na linha 20:



36 - Na Row podemos também definir o `mainAxisAlignment` para `spaceEvenly` (teste outros como `.center` e etc para ver a diferença). Vamos definir também uma altura (`height`) para cada imagem:



Código-fonte completo

main.dart

```
import 'package:flutter/material.dart';  
import 'package:jokenpo/jogo.dart';
```

```
void main() {  
  runApp(MaterialApp(  
    home: Jogo(),  
  ));  
}
```


jogo.dart

```
import 'package:flutter/material.dart';

class Jogo extends StatefulWidget {
  const Jogo({Key? key}) : super(key: key);

  @override
  State<Jogo> createState() => _JogoState();
}

class _JogoState extends State<Jogo> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
        title: const Text('JokenPO'),
      ),
      body: const Column(
        crossAxisAlignment: CrossAxisAlignment.center,
        children: <Widget>[
          //1 - Escolha do App
          Padding(
            padding: EdgeInsets.only(top: 32, bottom: 16),
            child: Text(
              "Escolha do App",
              textAlign: TextAlign.center,
              style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
            ),
          ),
          //2 - Imagem de escolha do app
```

```
Image(image: AssetImage('images/padrao.png'))),
```

```
//3 - Escolha uma opção abaixo:
```

```
Padding(
```

```
padding: EdgeInsets.only(top: 32, bottom: 16),
```

```
child: Text(
```

```
  "Escolha uma opção abaixo:",
```

```
  textAlign: TextAlign.center,
```

```
  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
```

```
),
```

```
),
```

```
Row(
```

```
mainAxisAlignment: MainAxisAlignment.spaceEvenly,
```

```
children: <Widget>[
```

```
  Image(
```

```
    image: AssetImage('images/papel.png'),
```

```
    height: 100,
```

```
  ),
```

```
  Image(
```

```
    image: AssetImage('images/papel.png'),
```

```
    height: 100,
```

```
  ),
```

```
  Image(
```

```
    image: AssetImage('images/tesoura.png'),
```

```
    height: 100,
```

```
  ),
```

```
],
```

```
)
```

```
],
```

```
),
```

```
);
```

```
}
```

```
}
```